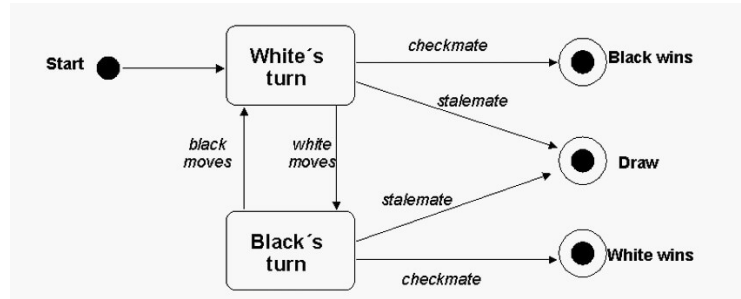


Design Patterns 1

Eindopdracht: Finite State Machine – viewer/simulator

Opdrachtomschrijving:

Binnen software-ontwikkeling wordt vaak gebruik gemaakt van Finite State Machines (FSM) om gedrag van een stuk software te modelleren. In UML gebruiken we toestandsdiagram (State Diagrams) om FSMs te modelleren. Deze beschrijft dan alle toestanden (*states*) waarin het systeem kan verkeren alsmede de overgangen (*transitions*) daartussen. Hier rechts zie je een eenvoudig toestandsdiagram voor de states waarin een schaakwedstrijd kan verkeren.



Voor Design Patterns 1 gaan we een eenvoudige FSM-viewer/simulator bouwen.

De viewer is in staat om uit een bestand een beschrijving van een FSM te lezen en deze middels objecten run-time te representeren. Met dit model van objecten kan de FSM tekstueel of grafisch weergegeven worden en eventueel worden gesimuleerd.

Indien je niet vertrouwd bent met FSMs kun je op deze sites wat achtergrondinformatie lezen.

<https://www.spiceworks.com/tech/tech-general/articles/what-is-fsm/>

https://en.wikipedia.org/wiki/Finite-state_machine

Een uiteenzetting van de elementen van de UML notatie die de FSM viewer moet ondersteunen vind je op de volgende site:

<https://sparxsystems.com/resources/tutorials/uml2/state-diagram.html>

De viewer hoeft enkel de basisconstructies ((compound/initial/final) states, (self)transitions, state actions, triggers/guard/effects) elementen te ondersteunen. Op de webpagina dus totaan het kopje "Entry point".

Beoordeling

Deze opdracht zullen jullie in duo's maken. Verderop in dit document staan de precieze requirements van de opdracht. Zie voor een gedetailleerde weging van de verschillende beoordelingscriteria ook de Rubric op Brightspace.

Lever op Brightspace de volgende onderdelen in:

- Domeinklassediagram, deadline: eind lesweek 4 (formatief)
 - o Geef aan waar de diverse Design Patterns die je gaat toepassen terugkomt in het diagram.
- Je uiteindelijke oplossing deadline eind week 8
Dit bevat onder andere:
 - o Applicatiecode
 - o Applicatieklassediagram van jouw uiteindelijke oplossing als documentatie van de oplossing
 - o Unit tests
 - o Zelf ingevulde rubric

Must-have requirements (voor cijfer tot 8)

Functioneel

- Ondersteun minimaal de volgende toestandsdiagram-onderdelen:
 - Initial en Final states
 - Simple state (met On Entry/Do/On Exit state actions)
 - Compound states (deze moeten meer dan 1 niveau diep genest kunnen worden)
 - Transitions (met bijhorende triggers, guards en effects). Je mag er vanuit gaan dat er maximal 1 guard en effect op een transition gemodelleerd kan worden.
 - Self-transitions
- De structuur (voor formaat zie bijlage A) van het toestandsdiagram moet ingelezen kunnen worden uit de bestanden die aangeleverd zijn op Brightspace.
- De toestandsdiagrammen moeten in de Console tekstueel kunnen worden weergegeven. Zie bijlage C voor een mogelijke weergave maar voel je vrij om zelf iets te kiezen. Het moet mogelijk zijn om ook delen van een diagram te tekenen. Naast het gehele diagram dus ook een enkele state met de transities van een naar die state, een compound state met sub-states en transities, een enkele transitie, etc. Dit is nodig voor een toekomstige uitbreiding waarbij het diagram ook moet kunnen worden aangepast. De gebruiker selecteert dan bijvoorbeeld een enkele state die dan los getekend moet kunnen worden zodat er wijzigingen kunnen worden aangebracht.
- Bij incorrecte toestandsdiagrammen moet de applicatie een foutmelding geven. Er dienen drie validators gemaakt te worden die de onderstaande fouten in het diagram kunnen detecteren (zie bijlage B voor nadere toelichting). Bij de 2^e en 3^e validator kun je zelf kiezen welke van de twee varianten je wilt bouwen.
 - State met transitions die niet deterministisch zijn
 - Initial state met ingaande transities **òf** Final state met uitgaande transities
 - Transitie naar een compound state i.p.v. naar een simple state binnen de compound state **òf** Onbereikbare state

Op Brightspace zijn een aantal bestanden met incorrecte FSM-definities te downloaden. Gebruik die in jouw tests om de validators te testen. Je hoeft niet zelf extra (exotische) ongeldige FSM's te genereren om te testen of een validator in alle gevallen goed werkt.

Niet-Functioneel

- Er wordt gelet op nette code en het juiste gebruik van verschillende design patterns. Er moeten dan ook verschillende design patterns toegepast worden en uitgelegd kunnen worden.
- De Presentatielaag (user interface) en Modellaag moeten strikt gescheiden zijn. De presentatielaag moet dus eenvoudig kunnen worden vervangen met een andere user interface. Deze eis geldt ook als je niet de extra requirement van een 2^e user interface implementeert.
- De opdracht moet gemaakt worden in Java of C#. Andere object georiënteerde talen zijn ook toegestaan na toestemming van jouw practicumdocent.

Nice to have requirements (voor cijfers 8-10)

Bovenstaande requirements geven je de mogelijkheid om een goed (8) te behalen. Indien je een hoger cijfer wil kun je één van onderstaande extra requirements implementeren. **Er wordt dus maar 1 nice-to-have beoordeeld. De *must-have requirements* moeten allemaal behaald zijn voordat er gekeken wordt naar de *nice-to-haves*. Het is niet toegestaan om beide nice-to-haves te implementeren om tekortkomingen bij de basis-eisen te compenseren. Zijn er toch 2 nice-to-haves gerealiseerd dan wordt maar een daarvan beoordeeld.**

Nice-to-have 1: Tweede user interface

Naast de tekstuele weergave kan het toestandsdiagram ook grafisch weergegeven worden. Dat kan door een GUI te maken maar bijvoorbeeld ook door de output naar een image bestand te schrijven zoals in de workshop-opdrachten wordt gedaan. Voor het tekenen van grafische elementen mag je gebruik maken van bestaande libraries. De uitbreiding gaat vooral over het realiseren van een tweede user interface waardoor de scheiding tussen Model en Presentation-lagen ook daadwerkelijk tot uiting komt.

De grafische weergave hoeft niet per sé een mooie layout te hebben (is best een uitdagend probleem) maar wel de verschillende toestanden met acties en welke transities ze hebben laten zien.

Mogelijk kun je het Abstract Factory patroon toepassen om het creëren van de user interface objecten makkelijk te kunnen wisselen tussen varianten voor tekstuele weergave en grafische weergave.

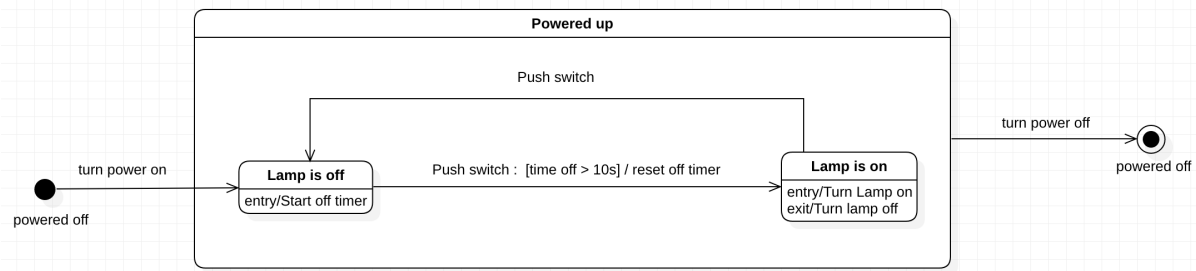
Nice-to-have 2: Simuleren van de FSM

Een tweede mogelijkheid is om aan de FSM-viewer ook de mogelijkheid van simulatie toe te voegen. Na het inlezen en (tekstueel) tonen van het FSM heeft de gebruiker dan de mogelijkheid om een simulatie te starten. De simulatie start uiteraard bij de *initial state*. De user interface wordt uitgebreid om in de tekstuele weergave van het FSM aan te kunnen geven in welke toestand de simulatie zich op dat moment bevindt. Vanuit de actuele toestand worden transities naar andere states gevolgd wanneer dat mogelijk is. Hiervoor kan het nodig zijn dat de gebruiker een bepaalde trigger activeert. De applicatie toont daarom een overzicht van de triggers in de FSM en de mogelijkheid voor de gebruiker om er één te activeren. Bij het activeren van een trigger kan de gebruiker eventueel een guard condition valideren indien dat nodig is. Na een transitie wordt de nieuwe toestand van het FSM getoond.

Het verloop van de simulatie moet netjes gelogd worden zodat duidelijk is welke *state transitions* hebben plaatsgevonden op basis van welke *trigger / guard conditions*. Ook *state actions* en *transition effects* die worden uitgevoerd moeten worden gelogd. Met deze log is de simulatie na afloop dus nog 'terug te lezen'.

Bijlagen

Bijlage A: Voorbeeld input file



```

# FSM1.txt
#
# Toggle Lamp. Deze file bevat een eenvoudige FSM
# voor een switch die een lamp aan en uit kan
# schakelen
#
#
#
# Description of all the states
#
STATE initial _ "powered off" : INITIAL;
STATE powered _ "Powered up" : COMPOUND;
STATE off powered "Lamp is off" : SIMPLE;
STATE on powered "Lamp is on" : SIMPLE;
STATE final _ "powered off" : FINAL;

#
# Description of all the triggers
#
TRIGGER power_on "turn power on";
TRIGGER push_switch "Push switch";
TRIGGER power_off "turn power off";

#
# Description of all the actions
#
ACTION on "Turn lamp on" : ENTRY_ACTION;
ACTION off "Turn lamp off" : EXIT_ACTION;
ACTION t2 "reset off timer" : TRANSITION_ACTION;

#
# Description of all the transitions
#
TRANSITION t1 initial off power_on;
TRANSITION t2 off on push_switch "time off > 10s";
TRANSITION t3 on off push_switch;
TRANSITION t4 powered final power_off;
  
```

De input files bestaan uit een aantal secties (states, triggers, actions, transitions) die altijd in deze volgorde in de tekstfile staan.

Elke regel in de specificatie file bestaat uit slechts één aparte definitie. Elke definitie wordt afgesloten met een “;”

Alles achter een ‘#’ teken tot aan het einde van de regel wordt gezien als commentaar. Commentaar kan niet op een regel voorkomen die al een definitie bevat.

Spaties en/of andere white-space karakters tussen regels zijn niet relevant.

Regels worden afgesloten met een line feed en vervolgens een carriage return karakter (LF en CR, oftewel \r\n) zoals gebruikelijk bij Windows ASCII tekstbestanden.

Als eerste worden alle *states* van het toestandsdiagram benoemd. Per regel wordt een enkele state gedefinieerd met als syntax:

STATE <identifier> <parent> " <name> " : <state_type> ;

- Hierin is *identifier* een string bestaande uit ten minste 1 karakter uit de verzameling {a-z, A-Z}. Deze *identifiers* zijn uniek binnen de *input file* en kunnen verderop in het bestand voorkomen om verbanden/koppelingen tussen verschillende onderdelen van het FSM aan te kunnen geven.
- Hierin is *parent* de *identifier* van de super-state die deze state bevat. Deze state(identifier) moet al eerder in het bestand zijn gedefinieerd. Indien er geen parent is staat hier een _ (*underscore*).
- Hierin is *name* een willekeurige string waar alle karakters m.u.v. " in mogen voorkomen.
- Hierin is *state_type* een element uit de verzameling { **INITIAL**, **SIMPLE**, **COMPOUND**, **FINAL** }. Deze geeft dus aan wat voor soort state het is.

TRIGGER <identifier> " <description> " ;

- Ook triggers hebben hun eigen *identifier* gevolgd door een beschrijving/weergave van de trigger tussen "".

ACTION <identifier> " <description> " : <action_type> ;

- Acties horen bij een state of transition. De *identifier* geeft aan bij welke STATE of TRANSITION deze actie hoort. De "eigenaar" wordt gevolgd door een beschrijving/weergave van de actie zelf tussen "".
- Hierin is *action_type* een element uit de verzameling { **ENTRY_ACTION**, **DO_ACTION**, **EXIT_ACTION**, **TRANSITION_ACTION** }. Deze geeft dus aan wat voor soort actie het is. De acties die plaatsvinden bij *transities* worden binnen UML *effects* genoemd.

TRANSITION <identifier> <source_state_identifier> -> <destination_state_identifier> [*<trigger_identifier>*] " <guard_condition> " ;

- Transitie hebben naast een eigen *identifier* ook altijd verwijzingen naar twee *state_identifiers* die eerder in het bestand al gedefinieerd zijn. De bron en doel states van de transitie zijn daarbij in het bestand gescheiden door de karakters "->".
- Na de *destination_identifier* volgt **optioneel** een *trigger_identifier*. Een transitie kan namelijk ook trigger-loos zijn. De zogenaamde *automatic transition*.
- Na de *identifiers* volgt nog een string met de guard condition van de transitie. Deze string is leeg indien er geen guard is. De twee "" tekens staan dan nog wel in het bestand.

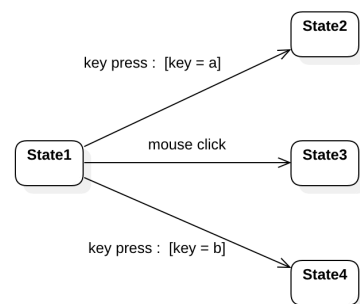
Bijlagen

Bijlage B: Toelichting fouten in UML state diagrams

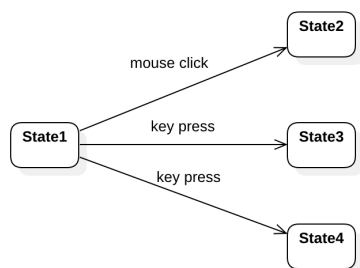
State met transitions die niet deterministisch zijn

Wanneer een state meerdere uitgaande transities heeft dan moet altijd duidelijk zijn welke transitie gevolgd moet worden. Dat wordt bepaald door de combinatie van *triggers* en *guard conditions*. Deze transities zijn dan deterministisch te noemen omdat ze volledig vastgelegd zijn met oorzaak/gevolg.

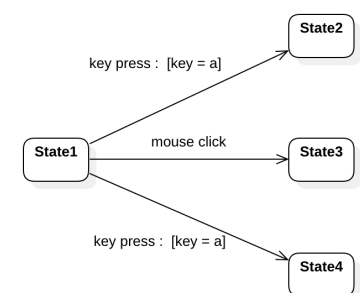
Het diagram rechts is valide. Voor alle drie de transities is duidelijk welke gevolgd moet worden in het geval een “mouse click” of “key press” trigger plaatsvindt.



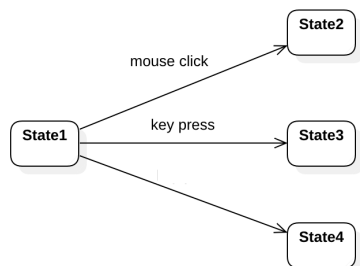
Het diagram rechts is **niet** valide. Het is onduidelijk wat er in het geval van een “key press” trigger moet gebeuren.



Het diagram rechts is ook **niet** valide. Het is onduidelijk wat er in het geval van een “key press” trigger moet gebeuren. De guards maken de transitities niet uniek.

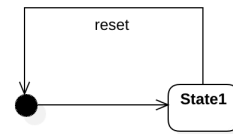


Het diagram rechts is ook **niet** valide. De automatic transitie maakt de overige transities onmogelijk.

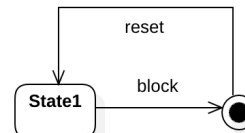


Initial state met ingaande transitie

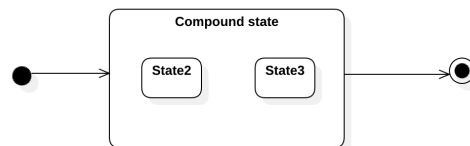
Het diagram rechts is **niet** valide. *Initial states* mogen geen ingaande transitie hebben.

**Final state met uitgaande transitie**

Het diagram rechts is **niet** valide. *Final states* mogen geen uitgaande transitie hebben.

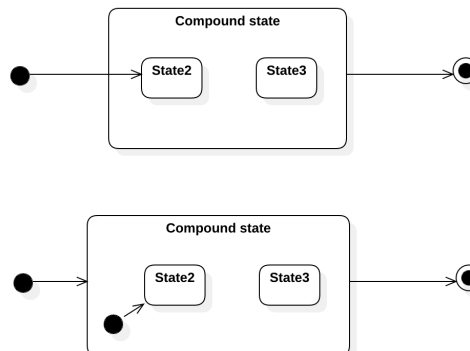
**Een transitie naar een compound state i.p.v. naar een simple state binnen de compound state**

Het diagram rechts is **niet** valide. Transitie kunnen niet eindigen bij een *compound state*.

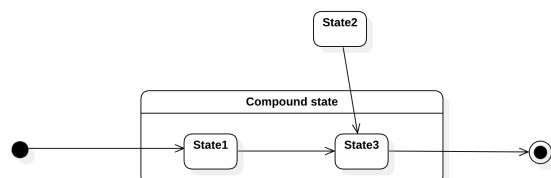


De twee diagrammen rechts zijn **wel** valide. Transitie eindigen bij een *sub state* van een *compound state* of vertrekken vanaf een *compound state*.

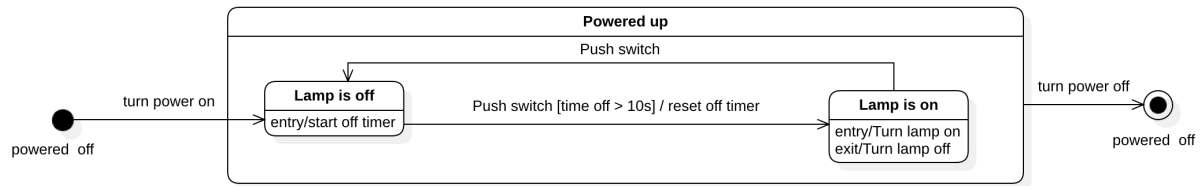
De notatie waarbij een transitie eindigt bij een *compound state* die zelf een *initial state* bevat hoeven jullie niet te ondersteunen.

**Onbereikbare state**

Het diagram rechts is **niet** valide. Er is geen mogelijkheid om in State2 te komen.



Bijlage C: Voorbeeld tekstuele weergave



Bovenstaand voorbeeld FSM kan bijvoorbeeld tekstueel als volgt worden weergegeven. Voel je echter vrij om een eigen weergave te bedenken/implementeren. In deze weergave is er een *indentation* (inspringen) gebruikt om subonderdelen beter herkenbaar te maken.

```

#####
# Diagram: Timed light
#####

0 Initial state (powered off)

--turn power on--> Lamp is off

=====
|| Compound state: Powered Up
-----

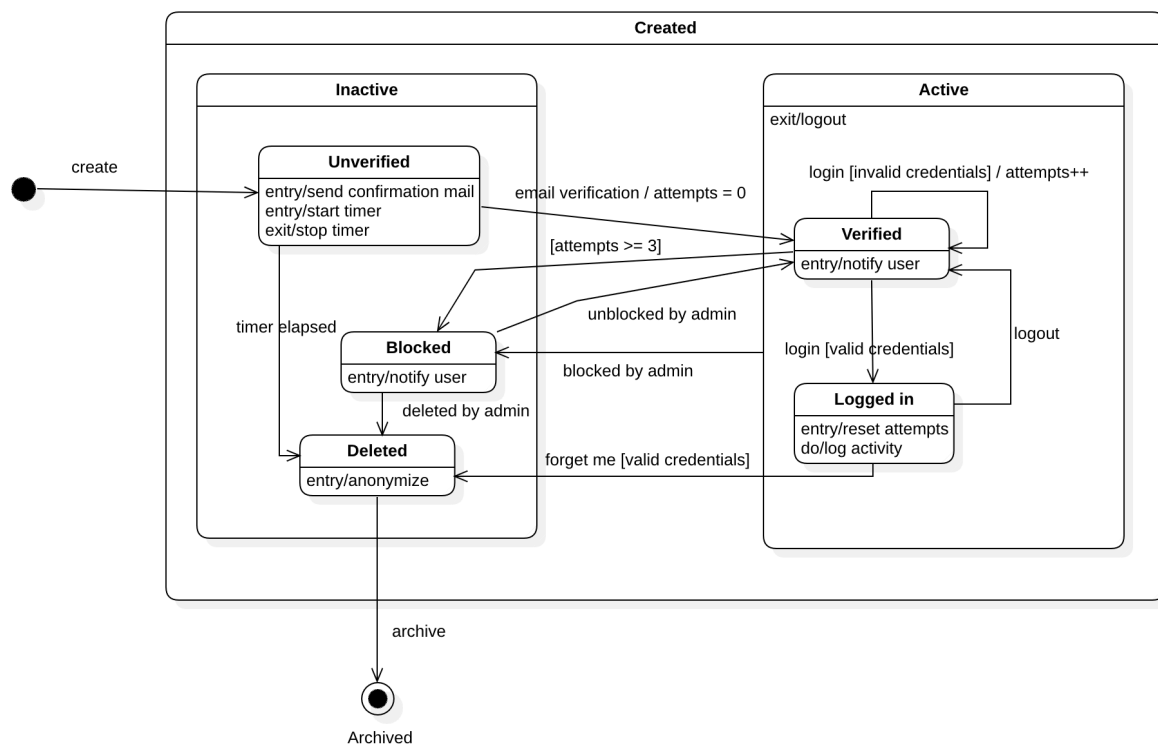
| Lamp is off
| On Entry / Start off timer
|-----
---Push switch [time off > 10s] / reset off timer---> Lamp is on

|-----
| Lamp is on
|-----
| On Entry / Turn lamp on
| On Exit / Turn lamp off
|-----
---Push switch---> Lamp is off

=====
---Turn power off---> Final state (powered off)

(0) Final state (powered off)
  
```

Bijlage D: Voorbeeld geneste compound states



Bovenstaand voorbeeld FSM voor een user account toont een wat complexer voorbeeld met geneste compound states. Je vindt het inputbestand voor deze FSM tevens op Brightspace.